Explainability-Driven Hallucination Detection Framework

for Offline Retrieval-Augmented Generation

"We are not building a chatbot. We are building a knowledge system that retrieves documents, explains why they were selected, and only then generates answers."

No Cloud APIs • No ChatGPT • No Black-Box Answers

M.Tech Main Project — Complete Technical Documentation

Department of Computer Science

Table of Contents

1. Project Overview & Problem Statement
2. Research Novelty & Contribution
3. Datasets & Knowledge Base Details
4. Literature Review Summary (70 Papers)
5. Complete System Architecture
6. Technology Stack Justification
7. Module-by-Module Implementation Details
    - 7.1 Ingestion Layer (Document Loader + Chunker)
    - 7.2 Retrieval Layer (Embedder + FAISS Store)
    - 7.3 Generation Layer (Local LLM via Ollama)
    - 7.4 Explainability Layer — Core Novelty
    - 7.5 Detection Layer (NLI + BERTScore + Cosine)
    - 7.6 Hallucination Engine (Score Aggregator)
    - 7.7 Decision Controller (Tri-State Output)
    - 7.8 Pipeline Orchestration
8. Hallucination Detection Algorithm
9. Streamlit UI Dashboard
10. Evaluation Framework & Metrics
11. Project File Inventory (All 26 Files)
12. Setup & Usage Guide
13. Research Objectives Mapping

1. Project Overview & Problem Statement

Title: Design and Development of an Explainability-Driven Hallucination Detection Framework for Offline Retrieval-Augmented Generation

Problem Statement

Large Language Models (LLMs) integrated with Retrieval-Augmented Generation (RAG) frameworks have significantly improved the factual grounding of generated responses. However, despite retrieval support, **hallucinations** — generated statements that are unsupported or weakly supported by retrieved evidence — remain a critical challenge, particularly in domain-specific and offline deployments.

Most existing RAG systems focus on improving retrieval quality or generation fluency, while hallucination detection is typically addressed using external fact-checking models, entailment classifiers, or post-hoc validation techniques. These approaches introduce **additional computational overhead**, reduce interpretability, and are **unsuitable for offline or resource-constrained environments**.

Research Question

How can explainability signals generated within an offline RAG framework be systematically leveraged to detect, suppress, or control hallucinated outputs, while maintaining transparency, efficiency, and evidence grounding?

Key Challenges Addressed

- Generated responses may include claims not fully supported by retrieved documents, even when retrieval quality is high.
- Existing hallucination detection methods rely on heavyweight models or external services, limiting offline applicability.
- RAG systems lack mechanisms to quantify and explain the degree of evidence support for individual generated statements.
- Insufficient integration between retrieval explainability and generation reliability within a unified framework.

2. Research Novelty & Contribution

Most RAG systems follow: **Retrieve** → **Generate** → **(maybe) Check**

This system follows: **Retrieve** → **Score** → **Generate** → **Detect using those same scores** → **Decide**

The explainability signals are **not an afterthought** — they are built into the pipeline from the start, making hallucination detection **intrinsic, transparent, and offline-capable**. This is the core research contribution.

Aspect	Existing Approaches	Our Approach
Detection Method	External fact-checkers, post-hoc NLI	Internal XAI signals used as primary features
Offline Capability	Most need cloud APIs / internet	Fully offline, all models run locally
Explainability	Treated as separate module	Integrated into detection pipeline
Output Control	Binary (pass/fail)	Tri-state: Cite / Warn / Block
Evidence Tracing	Limited or absent	Per-sentence source attribution + provenance
Computational Cost	Heavy (external models)	Lightweight (reuses retrieval signals)

3. Datasets & Knowledge Base Details

This project uses **two categories of data**: (A) the document corpus that the RAG system indexes and retrieves from, and (B) the literature review dataset of 70 research papers.

3.1 Document Corpus (Knowledge Base)

The system indexes domain-specific documents placed in the `data/corpus/` directory. For this project, **9 documents** are used covering RAG systems, hallucination detection, and the project's own technical specifications:

Document Name	Type	Size	Content & Coverage
A Review of Faithfulness Metrics for Hallucination Assessment in LLMs	PDF	1,943 KB	Systematic review of 15+ faithfulness metrics (ROUGE, BLEU, BERTScore, EM, LM, QA-based). Compares LLM-based evaluators, NLI-based metrics, and n-gram overlap metrics. Finds LLM-based evaluators correlate most with human judgment.
Domain-Adapted RAG for Technical Documentation: Reliability & Faithfulness	PDF	950 KB	Domain-adapted RAG pipeline using BGE-M3 embedder fine-tuned with triplet loss. Achieves Recall@5: 0.67→0.86 improvement. Uses LLM-Critic for flagging unreliable queries. Demonstrates domain adaptation improves retrieval in technical documentation.
Enhancing RAG Pipelines with Agent-Based Hallucination Detection & Guardrails	PDF	1,441 KB	Guardrails via prompt concatenation and Langchain custom evaluator. TLM fine-tuning on Llama 3.2 3B achieves 79.16% accuracy, F1=79.21%. Agentic RefChecker pipeline for fact verification. Tested on Apple 10K financial domain.
Hallucination Detection in LLM Generated Texts: A Survey of Internal & External	PDF	782 KB	Comprehensive taxonomy: Internal methods (uncertainty/semantic entropy, token probability, hidden state analysis) vs. External methods (retrieval, NLI, knowledge graphs). GPT-4 achieves ~82% hallucination detection rate on HaluEval.
Incorporating RAG for Factual Hallucination Detection in Intelligent Educational Systems	PDF	1,420 KB	RAG pipeline with Sentence-BERT vectorization + FAISS approximate nearest neighbour search. Outperforms ChatGLM3-6B baseline across all 5 text types. Applied to educational content verification for factual accuracy in teaching materials.
Problem Statement	PDF	112 KB	Defines the research problem, objectives, scope, and novelty of this project. Establishes that the system uses internal XAI signals for hallucination control, not external fact-checkers. Lists 5 project objectives.
RAG Hallucination Framework Overview	PDF	39 KB	Visual system architecture diagram showing the complete pipeline flow: User query → Retriever → Explainability Layer → LLM Generator → Hallucination Detector → Tri-state output (Answer/Partial/Block).
Tech Stack Summary	PDF	38 KB	Tabular comparison of technology choices with justifications: all-MiniLM-L6-v2 (embedding), FAISS (search), Mistral/LLaMA (generation), DeBERTa (NLI), BERTScore (semantic), LangChain (orchestration).
Hallucination Control in Offline RAG Using NLI Entailment (Technical Guide)	DOCX	21 KB	Detailed technical guide explaining each pipeline component: Document Store, Embedder, Retriever, Explainability Layer (5 signal types), LLM Generator, Hallucination Detector (3 techniques), Decision Controller (3 states). Includes technology comparison tables with rejection rationale.

Corpus Statistics

- Total documents: **9**
- Total corpus size: **~6.7 MB**
- File formats: PDF (8 files), DOCX (1 file)
- Domain: RAG systems, hallucination detection, technical documentation
- After chunking (512 tokens, 50 overlap): ~200+ text chunks indexed in FAISS

Chunking Strategy

Documents are split into **512-token chunks** with **50-token overlap** between consecutive chunks. This overlap ensures that sentences spanning chunk boundaries are not lost. Each chunk retains metadata: `{chunk_id, source_file, page_num, token_count}`.

3.2 Literature Review Dataset (70 Research Papers)

A comprehensive survey of **70 research papers** stored in `research_papers.xlsx` with structured metadata.

Dataset Overview

Attribute	Value
Total Papers	70
Year Range	2018 – 2026
Conference Papers	40 (57%)
Journal Papers	30 (43%)
Unique Venues	49
Fields Per Paper	S.No, Title, Authors, Year, Venue, Type, Source Database, DOI/URL, Methods, Results, Conclusion, Research Gap, Limitations

Papers by Year Distribution

Year	Count	Percentage	Notable Trend
2018	1	1.4%	FEVER dataset (foundational benchmark)
2019	2	2.9%	Early RAG concepts
2020	6	8.6%	Original RAG paper (Lewis et al.), FactCC
2021	2	2.9%	Continued RAG development
2022	13	18.6%	TruthfulQA, InstructGPT, hallucination awareness surge
2023	16	22.9%	HaluEval, FActScore, SelfCheckGPT, survey papers
2024	24	34.3%	Peak — RAGAs, Semantic Entropy (Nature), domain-adapted RAG
2025	4	5.7%	Agent-based detection, guardrails mechanisms
2026	2	2.9%	Latest domain-adapted RAG, comprehensive surveys

Key Venues Represented

- ACL, EMNLP, NAACL, EACL (top NLP conferences)
- NeurIPS, ICML, ICLR (top ML conferences)
- Nature, AAAI, ACM Computing Surveys (high-impact journals)
- IEEE Transactions, Springer, Elsevier (engineering journals)
- arXiv preprints (cutting-edge research)

Key Papers by Category

RAG Foundations

- **Lewis et al., 2020**: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — Original RAG paper
- **Krainovskikh et al., 2026**: Domain-Adapted RAG for Technical Documentation — BGE-M3 fine-tuning

Hallucination Detection

- **Rahman & Chowdhury, 2026**: Survey of Internal and External Approaches — Comprehensive taxonomy
- **Li et al., 2023**: HaluEval benchmark — 35K examples, GPT-4 achieves 82% detection
- **Farquhar et al., 2024**: Semantic Entropy (Nature) — AUROC 0.89 for detection
- **Manakul et al., 2023**: SelfCheckGPT — Zero-resource black-box detection via consistency

Evaluation Metrics

- **Malin et al., 2025**: Review of Faithfulness Metrics — 15+ metrics compared
- **Min et al., 2023**: FActScore — Atomic fact-level evaluation
- **Es et al., 2024**: RAGAs — Automated RAG evaluation framework

Fact Verification & NLI

- **Thorne et al., 2018**: FEVER — 185K claims benchmark for fact verification
- **Kryscinski et al., 2020**: FactCC — NLI-based factual consistency for summarization

Applied RAG + Hallucination Control

- **Zheng et al., 2025**: RAG for Educational Systems — Sentence-BERT + FAISS
- **Mishra et al., 2025**: Agent-Based Hallucination Detection — Guardrails + TLM

Research Gaps Identified Across Literature

- Most systems require cloud APIs / internet — **not suitable for offline/edge deployment**
- Hallucination detection is treated as an **independent post-hoc module**, disconnected from retrieval
- Explainability signals (similarity, attention, provenance) are **rarely used as detection features**
- Output control is typically binary (pass/fail) — **no graduated response with partial answers**
- Cross-domain generalization is weak — **most tested on single domains only**
- Limited integration of multiple detection techniques — **NLI, BERTScore, cosine used independently**

5. Complete System Architecture

6-Stage Pipeline Flow

Stage	Component	Input	Output	Model/Tool
1	Document Ingestion	PDF, DOCX, TXT files	Page-level text records with metadata	PyPDF2, python-docx
2	Text Chunking	Page text records	512-token chunks (50 overlap) with source tracking	Custom sliding-window
3	Embedding + Indexing	Text chunks	384-dim L2-normalized vectors + FAISS index	all-MiniLM-L6-v2 + FAISS
4	Query + Retrieval	User question	Top-K chunks ranked by cosine similarity	FAISS IndexFlatIP
5	LLM Generation	Query + retrieved chunks	Context-grounded answer text	Mistral 7B via Ollama
6a	XAI Layer ★	Query, chunks, answer	Similarity scores, attribution, provenance, heatmaps	Embedder + custom logic
6b	NLI Detection	Sentence + best chunk	Entailment / Neutral / Contradiction probabilities	DeBERTa NLI cross-encoder
6c	BERTScore Detection	Sentence + chunks	Semantic overlap F1 score (0–1)	distilbert-base-uncased
6d	Score Aggregation	NLI + BERTScore + Cosine	Weighted support score per sentence	0.45×NLI + 0.35×BERT + 0.20×Cos
7	Decision Controller	Support scores + summary	CITE ■ / WARN ■ / BLOCK ■	Threshold-based tri-state logic

6. Technology Stack Justification

Layer	Tool / Library	Why Chosen	Alternatives Rejected & Reason
Embedding	all-MiniLM-L6-v2	Offline, fast, 80MB, 384-dim, excellent semantic similarity	OpenAI ada (needs internet), BERT-base (slower, not optimized for similarity)
Vector Search	FAISS (CPU)	Runs fully in RAM, blazing fast, no server, battle-tested at scale	Elasticsearch (too heavy), ChromaDB (slower), Pinecone (cloud-only)
LLM Generator	Mistral 7B / LLaMA 3	Open-source, runs offline via Ollama, strong generation quality	GPT-4/ChatGPT (cloud, costs money), BERT (not generative), T5 (weaker)
NLI Entailment	DeBERTa NLI cross-encoder	Best entailment accuracy, lightweight, offline, 3-class output	RoBERTa-NLI (less accurate), GPT-based NLI (needs API)
Semantic Score	BERTScore (distilbert)	Token-level semantic overlap, no external calls, F1/P/R output	ROUGE (surface overlap only), BLEU (n-gram matching, misses semantics)
Orchestration	Custom Python	Full control, modular, no unnecessary abstractions	LangChain (added complexity for this use case)
UI Dashboard	Streamlit + Plotly	Rapid interactive dashboard, dark theme, rich charts	Flask (more boilerplate), Gradio (less customizable)

7. Module-by-Module Implementation Details

7.1 Ingestion Layer

`src/ingestion/`

document_loader.py (4.2 KB)

- Loads PDF files using PyPDF2 — extracts text page-by-page with page numbers
- Loads DOCX files using python-docx — extracts all paragraphs
- Loads TXT/MD files via plain text reading
- Auto-detects file type by extension
- load_corpus() walks a directory recursively and loads all supported files
- Each record: {text, source_file, page_num, file_type}

chunker.py (4.2 KB)

- Sliding-window token-based chunker: 512 tokens per chunk, 50-token overlap
- Each chunk gets a UUID chunk_id for tracking
- Metadata preserved: {chunk_id, text, source_file, page_num, token_count, start_word, end_word}
- get_sentences() function splits generated answers into individual claims for detection
- Regex-based sentence splitting (no external NLTK server needed)

7.2 Retrieval Layer

`src/retrieval/`

embedder.py (3.7 KB)

- Wraps sentence-transformers all-MiniLM-L6-v2 model
- Produces 384-dimensional L2-normalized vectors
- L2 normalization means dot product = cosine similarity (efficient)
- Batch encoding with configurable batch_size (default 64)
- embed_query() for single queries, embed_chunks() for corpus indexing

faiss_store.py (7.4 KB)

- FAISS IndexFlatIP — exact inner product search (= cosine when L2-normalized)
- build(chunks, embeddings) → creates index and persists to disk
- search(query_vec, k) → returns top-K results with similarity scores and metadata
- Saves index as faiss.index + chunks_metadata.pkl in data/index/
- load() restores index from disk for fast restarts without re-embedding

7.3 Generation Layer

`src/generation/`

llm_generator.py (7.7 KB)

- Connects to local Ollama server at http://localhost:11434
- Supports any Ollama model: mistral, llama3, phi3, gemma2
- Builds grounded prompts: system instructions + context chunks + query
- System prompt enforces: 'Answer ONLY using provided context, do NOT fabricate'
- Auto-detects if Ollama is running at startup
- Fallback mode: when Ollama is offline, returns top chunk text as extractive answer
- This fallback allows the entire detection pipeline to be tested without a GPU

7.4 Explainability Layer ★ (Core Novelty)

`src/explainability/`

xai_layer.py (11.9 KB)

- ★ This is the heart of the project's research contribution
- 5 explainability functions that capture WHY the system gave each answer:

■ 1. compute_similarity_scores() — cosine similarity per chunk to the query

■ 2. compute_chunk_attribution() — maps each generated sentence to its best-matching source chunk using embedding similarity

■ 3. compute_provenance_score() — fraction of sentences traceable to documents (grounding measure)

■ 4. build_similarity_heatmap() — structured data for UI bar chart visualization

■ 5. compute_word_saliency() — word-level lexical overlap proxy for attention highlighting

- generate_xai_report() combines all 5 into a complete explainability report
- These signals are used DIRECTLY as hallucination detection features — not added after the fact

7.5 Detection Layer

`src/detection/`

nli_detector.py (8.3 KB)

- Loads cross-encoder/nli-deberta-v3-small locally
- Scores each (chunk, sentence) pair → {ENTAILMENT, NEUTRAL, CONTRADICTION} probabilities
- Catches logical contradictions: 'X causes Y' when chunk says 'Y prevents X'
- Has word-overlap heuristic fallback when the model is unavailable
- score_sentences() processes all claims against their attributed chunks

bertscore_detector.py (7.2 KB)

- Uses bert_score library with distilbert-base-uncased model
- Computes token-level semantic F1 between generated sentence and source chunks
- Catches semantic drift: different topic, different meaning even if words overlap
- Returns {bertscore_f1, bertscore_precision, bertscore_recall, best_reference_idx}
- Falls back to TF-IDF cosine similarity when bert-score is not installed

hallucination_engine.py (9.6 KB)

• Aggregates all three detection scores into one support score per sentence:

■ support_score = 0.45 × NLI_entail + 0.35 × BERTScore_F1 + 0.20 × cosine

• Classifies: SUPPORTED (≥0.75), WEAKLY_SUPPORTED (≥0.50), HALLUCINATED (<0.50)

• SentenceVerdict dataclass stores complete per-claim evidence

• summarize() produces aggregate stats for the decision controller

7.6 Decision Controller

`src/controller/`

decision_controller.py (12.4 KB)

• Tri-state output logic based on aggregated support scores:

■ STATE 1 (≥0.75): Full answer with inline citations [source.pdf, p.3]

■ STATE 2 (≥0.50): Partial answer — supported claims shown, weak claims flagged ■, hallucinated claims suppressed ■

■ STATE 3 (<0.50 or >50% hallucinated): Entire answer blocked, user asked to rephrase

• Additional safety rule: blocks if >50% of claims are hallucinated, regardless of average score

• format_for_display() produces human-readable output with emoji indicators

7.7 Pipeline Orchestration

`src/`

pipeline.py (16.8 KB)

• RAGHallucinationPipeline class wires all 6 stages together

• index_documents() — one-time: load → chunk → embed → build FAISS index

• query() — repeated: embed query → retrieve → generate → XAI → detect → decide

• Returns PipelineResult with all intermediate data for the UI

• Reads all settings from config/config.yaml

• CLI mode: 'python src/pipeline.py query' for interactive terminal queries

8. Hallucination Detection Algorithm

For every user query, the system executes this 9-step algorithm:

Step 1: Embed Query

Convert the user's question into a 384-dim vector using all-MiniLM-L6-v2.

Step 2: Retrieve Top-K Chunks

Find the 5 most similar document chunks using FAISS cosine search.

Step 3: Compute Similarity Scores

Record cosine similarity for each chunk — these become explainability signals.

Step 4: Generate Answer

Feed query + top-K chunks into Mistral/LLaMA via Ollama local server.

Step 5: Sentence-Split

Break the generated answer into individual claims/sentences.

Step 6: Run NLI on Each Claim

For each claim, check: does the best-matched chunk ENTAIL this claim?

Step 7: Compute BERTScore

Measure semantic overlap F1 between each claim and source chunks.

Step 8: Aggregate Confidence

Combine: support = 0.45×NLI + 0.35×BERTScore + 0.20×cosine.

Step 9: Apply Decision Threshold

Map score to: ■ CITE (≥0.75) / ■ WARN (≥0.50) / ■ BLOCK (<0.50).

Why Three Detectors Together?

Detector	What It Catches	Example
NLI Entailment	Logical contradictions	Claim says 'X causes Y' but chunk says 'Y prevents X'
BERTScore	Semantic drift	Claim about climate change when chunk is about RAG systems
Cosine Similarity	Out-of-domain claims	Nothing in retrieved docs is semantically close to the claim

9. Streamlit UI Dashboard

The interactive dashboard (`ui/app.py`, 28.1 KB) provides a premium dark-themed interface built with Streamlit and Plotly:

UI Component	Description
Sidebar Controls	Model selection (embedding, LLM, NLI), top-K slider, threshold sliders, score weight sliders, index build button
Query Input	Text input with one-click Query button
Score Cards	5 top-level metrics: Decision State badge, Grounding Score %, Provenance %, Hallucinated Claims count, Sources Cited count
Color-Coded Answer	Each sentence displayed with ■ green (supported), ■ amber (weak), or ■ red (hallucinated) borders and badges
Similarity Bar Chart	Plotly horizontal bar chart showing per-chunk cosine similarity to the query, color-coded green→red
Provenance Gauge	Plotly gauge indicator showing what fraction of the answer is traceable to documents
Radar Chart	Per-claim component score breakdown (NLI, BERTScore, Cosine, Aggregate) as overlaid polar plots
Detection Log Table	Full table with columns: #, Claim, NLI Score, NLI Label, BERTScore F1, Cosine, Support Score, Verdict, Source
Chunk Inspector	Expandable sections showing each retrieved chunk's text, source, and similarity score
Query History	Table tracking all queries with their decision state, score, and response time

10. Evaluation Framework & Metrics

Hallucination Detection Metrics

Metric	Formula / Description	What It Measures
Precision	TP / (TP + FP)	Of predicted hallucinations, how many are truly hallucinated?
Recall	TP / (TP + FN)	Of actual hallucinations, how many did we catch?
F1 Score	2 × P × R / (P + R)	Harmonic mean balancing precision and recall
Accuracy	(TP + TN) / Total	Overall correctness of classification
AUC-ROC	Area under ROC curve	How well scores discriminate hallucinated from supported claims

Retrieval Quality Metrics

Metric	Description
Recall@K	Fraction of relevant documents found in top-K retrieved
MRR	Mean Reciprocal Rank — 1/rank of first relevant result
NDCG@K	Normalized Discounted Cumulative Gain — position-aware retrieval quality

Built-in Demo Test Set: 5 questions, 20 labeled claims (15 supported + 5 hallucinated) for quick validation.

Generated Outputs: ROC curve, score distribution histogram, metrics bar chart, JSON evaluation report — all saved to `evaluation/results/`

11. Project File Inventory

File Path	Size	Purpose
config/config.yaml	3.7 KB	Central configuration — thresholds, model names, paths
requirements.txt	0.4 KB	Python dependencies (23 packages)
README.md	8.8 KB	Complete setup, architecture, and usage guide
src/__init__.py	—	Package initializer
src/ingestion/__init__.py	—	Package initializer
src/ingestion/document_loader.py	4.2 KB	PDF/DOCX/TXT loading with metadata
src/ingestion/chunker.py	4.2 KB	512-token sliding-window chunker
src/retrieval/__init__.py	—	Package initializer
src/retrieval/embedder.py	3.7 KB	all-MiniLM-L6-v2 embedding wrapper
src/retrieval/faiss_store.py	7.4 KB	FAISS index build/search/persistence
src/generation/__init__.py	—	Package initializer
src/generation/llm_generator.py	7.7 KB	Ollama LLM wrapper with fallback
src/explainability/__init__.py	—	Package initializer
src/explainability/xai_layer.py	11.9 KB	★ Core XAI: 5 explainability functions
src/detection/__init__.py	—	Package initializer
src/detection/nli_detector.py	8.3 KB	DeBERTa NLI cross-encoder scoring
src/detection/bertscore_detector.py	7.2 KB	BERTScore semantic overlap
src/detection/hallucination_engine.py	9.6 KB	3-score aggregator + classifier
src/controller/__init__.py	—	Package initializer
src/controller/decision_controller.py	12.4 KB	Tri-state output logic
src/pipeline.py	16.8 KB	End-to-end pipeline orchestration
ui/app.py	28.1 KB	Streamlit interactive dashboard
evaluation/__init__.py	—	Package initializer
evaluation/metrics.py	10.4 KB	P/R/F1, AUROC, MRR, NDCG, plotting
evaluation/evaluator.py	14.1 KB	Test set evaluation runner

Total: 25 source files | All 23 Python files pass syntax validation ■

12. Setup & Usage Guide

Step 1: Install Python Dependencies

```
pip install -r requirements.txt
```

Installs all 23 packages: sentence-transformers, faiss-cpu, transformers, torch, bert-score, streamlit, plotly, etc.

Step 2: Install Ollama (Optional, for LLM generation)

```
ollama pull mistral ollama serve
```

Downloads and runs Mistral 7B locally. The system works without Ollama too — uses extractive fallback.

Step 3: Add Documents to Corpus

```
Copy files to data/corpus/
```

Place PDF, DOCX, or TXT domain documents. 9 documents are already included.

Step 4: Launch Dashboard

```
streamlit run ui/app.py
```

Opens the interactive web dashboard in your browser.

Step 5: Build Index

```
Click 'Build Index' in sidebar
```

Loads, chunks, embeds, and indexes all corpus documents into FAISS.

Step 6: Query

```
Type question → click 'Query'
```

Full pipeline runs: retrieve → generate → detect → decide. Results shown with color-coded verdicts.

13. Research Objectives Mapping

How each stated project objective is addressed by the implementation:

Objective	Implementation	Key Module(s)
Design and implement an offline RAG framework for domain-specific knowledge retrieval	FAISS + all-MiniLM-L6-v2 + Ollama local LLM. No internet required after model download.	embedder.py, faiss_store.py, llm_generator.py
Develop an explainability layer providing evidence attribution and confidence scores	5 XAI functions: cosine scores, chunk attribution, provenance, heatmaps, word saliency.	xai_layer.py (★ Core Novelty)
Propose an explainability-driven hallucination detection mechanism	3 detection techniques (NLI + BERTScore + Cosine) using XAI signals as primary features.	nli_detector.py, bertscore_detector.py, hallucination_engine.py

Objective	Implementation	Key Module(s)
Control hallucinated outputs by suppressing, providing partial answers, or requesting clarification	Tri-state decision controller: Cite (≥0.75) / Warn (≥0.50) / Block (<0.50).	decision_controller.py
Experimentally evaluate using hallucination detection metrics and retrieval accuracy	P/R/F1, AUROC, Recall@K, MRR, NDCG. Demo test set with 20 labeled claims. Plot generation.	metrics.py, evaluator.py

---

This document was auto-generated from the project source code and configuration files.